

Obsidian 考研知識樹

Backbone + Card 學習系統使用說明書

目的不是建立一套漂亮的筆記，而是建立一張「我現在會什麼、哪裡有洞、下一步該做什麼」的能力地圖。

核心原則

章節是 Backbone，考點是 Card，細節是 Bullet，題目決定是否長出新的 Card。

1. 這套系統要解決什麼？

一般筆記容易變成「讀過很多、整理很多」，但很難直接回答：這個考點我到底會不會？考研需要的不是保存所有內容，而是快速定位弱點。

因此 Obsidian 在這套方法裡不是百科全書，而是**考點掌握度地圖**。讀書建立骨架，刷題驗證骨架；錯題暴露缺口後，再補上新的枝幹。

2. 整體資料夾架構

```
弟弟考試/  
├── 00 Dashboard/  
│   └── 考研總覽.md  
├── 01 Backbone/  
│   ├── 計算機組織.md  
│   ├── 作業系統.md  
│   ├── 資料結構.md  
│   ├── 演算法.md  
│   ├── 離散數學.md  
│   └── 線性代數.md  
├── 02 Cards/  
│   ├── CO/  
│   ├── OS/  
│   ├── DS/  
│   ├── Algorithm/  
│   ├── Discrete/  
│   └── Linear Algebra/  
├── 03 Review/  
│   └── 待補洞.md  
├── 90 Templates/  
│   └── 考點 Card.md  
└── 99 Attachments/
```

Folder 管收納，Link 管知識關係。不需要再建立 CO/Ch1/Ch2/Ch3 等大量資料夾。Card 放在哪一章，應由 [[Link]] 和 Backbone 決定。

3. Backbone：知識樹的主幹

每一科建立一份 Backbone Note。它不是完整筆記，而是導航頁，只保留章節以及重要考點的連結。

```
# 計算機組織  
  
## Instruction Set  
- [[MIPS]]  
  - [[MIPS Addressing]]
```

- [[Instruction Format]]
- [[Branch]]
- [[Procedure Call]]

Processor

- [[Pipeline]]
- [[Data Hazard]]
- [[Control Hazard]]

Memory

- [[Cache]]
- [[Cache Mapping]]
- [[Cache Miss]]
- [[AMAT]]
- [[Virtual Memory]]

Backbone 不需要一開始就完整。今天只知道 MIPS 和 Cache，就只放這兩個。之後刷到 Pipeline，再讓 Processor 這個枝幹長出來。

4. Card：一個可以獨立被考的考點

不是每個名詞都需要一張 Card。只有當一個概念可以獨立出題、反覆出現，或已成為你的弱點時，才值得升級成 Card。

```
---
subject: C0
chapter: Instruction
status: yellow
---

# MIPS Addressing

Parent: [[MIPS]]

## 核心
- Memory 是 byte addressing
- word = 4 bytes
- Address = Base + Offset

## 我容易錯
- array index 忘記乘 4
- offset 單位搞錯

## 延伸
- [[Array Address]]
- [[Load Store]]
```

Card 應保持短小。如果只需要一句話說清楚，就留在父 Card 的 Bullet 裡；當它變成獨立考點時，再拆成新 Card。

5. 掌握度狀態

狀態	代表	下一步
RED	觀念不會 / 無法解釋	讀觀念 + 基礎題
YELLOW	觀念懂，但題目不穩	不要重讀，直接刷題
GREEN	可以獨立解題	偶爾用題目驗證
MASTERED	隔一段時間仍穩定	退出主動複習

最重要的是 YELLOW：它表示「不是再讀一次，而是需要靠題目把能力練穩」。狀態不是永久的；做題再次暴露問題時，可以從 GREEN 或 MASTERED 降回 YELLOW / RED。

建議升級條件：RED → 能做基本題 → YELLOW → 不看筆記連續做對 2-3 題 → GREEN → 隔幾天再次遇到仍能獨立做對 → MASTERED。

6. 什麼時候新增 Card？

情況 A：可以獨立被考。 例如 Pipeline、Cache、Virtual Memory。

情況 B：刷題後發現反覆出現。 原本 Cache Miss 只是 [[Cache]] 裡的一個 Bullet，題目大量出現後，再拆成 [[Cache Miss]]。

情況 C：成為你的弱點。 原本 Array Address 只是 MIPS Addressing 的細節，但連續錯題後，就建立 [[Array Address]] Card。

不要因為課本有一個小標題，就自動建立 Card。讓題目決定你的知識樹需要長多細。

7. 錯題怎麼處理？

原則：**不要建立龐大的錯題庫。** 錯題的價值在於告訴你「哪個考點有洞」，而不是保存題目本身。

刷題做錯

↓

先判斷原因

├─ 粗心 / 單純算錯 → 不記

└─ 知識有洞 → 找對應 Card

- └─ Card 已存在 → 更新「我容易錯」+ status
- └─ Card 不存在 → 判斷是否值得新增 Card

例如 Array Address 題目做錯，不需要把整題抄進 Obsidian，只需要在 Card 加上「word index 忘記乘 4」等真正造成錯誤的規則。

8. 待補洞頁面

03 Review/待補洞.md 是你每天決定讀什麼的入口。初期可以完全手動維護。

```
# 待補洞

## RED - 不會
- [[Procedure Call]]
- [[Virtual Memory]]

## YELLOW - 需要刷題
- [[Array Address]]
- [[Cache Mapping]]
- [[AMAT]]

## GREEN - 已會
- [[MIPS Addressing]]
```

9. 每天實際怎麼使用？

Step 1 - 先刷題。 不要先打開 Obsidian 找東西整理。

Step 2 - 做錯才診斷。 問自己：是粗心、忘記，還是真的不懂？

Step 3 - 回到對應 Card。 如果知識有洞，補一兩條「核心」或「我容易錯」。

Step 4 - 更新 status。 不會就 RED；懂但題目不穩就 YELLOW。

Step 5 - 必要時讓樹長大。 一個細節開始反覆出題或變成弱點，才從 Bullet 升級成 Card。

Step 6 - 下一次讀書從 RED / YELLOW 開始。 MASTERED 不主動重讀。

你的循環應該是：**Backbone → 刷題 → 發現洞 → Card → 補洞 → 再刷題。**

10. 現階段不要做的事

1. 不要為了完整，把整本課本一次建成數百張 Card。
2. 不要把每道錯題完整抄進 Obsidian。

3. 不要花大量時間美化 CSS、圖示、Dashboard。
4. 不要因為 YELLOW 就重新讀完整章節；YELLOW 的主要工作是刷題。
5. 不要追求知識樹看起來完整。它應該反映你真正碰過、真正需要掌握的考點。

11. 什麼時候再加入 Dataview ？

先只使用 Folder + Markdown + [[Link]] + Properties + Templates。等累積約 50-100 張 Card、手動整理 RED / YELLOW 開始變麻煩，再導入 Dataview 自動產生 Dashboard。

12. 最終判斷規則

章節 = Backbone Note

考點 = Card

小知識 = Card 裡的 Bullet

錯題 = 用來修改 Card，而不是另一套筆記

題目 = 決定知識樹往哪裡生長

如果你開始花在「整理系統」上的時間比刷題還多，就代表 Card 拆得太細或紀錄得太多。這套系統的成功標準不是筆記漂亮，而是你能更快知道：**現在最值得補的是哪一個考點。**